UNIVERSITY OF MORATUWA

Faculty of Information Technology

IN1621 – Web Technologies

Homework

2 hours March 2026

Learning Objectives

By the end of this lab session, you will be able to,

1. Create a REST API using Node.js and Express
2. Implement Data Access Layer (DAL) pattern
3. Connect Node.js backend to MySQL database
4. Test API endpoints using Postman
5. Understand layered architecture basics

1. Installation

1. Download and install Node.js from https://nodejs.org/
   a. Verify installation
   ```
   node --version npm
   --version
   ```

2. Download and install MySQL from https://dev.mysql.com/downloads/installer/ 3. Download and install Postman from https://www.postman.com/downloads/

2. Database Creation

1. Start MySQL Workbench
2. Select the MySQL server using your password
3. Create a database named “library_db”
4. Create a table inside the library_db called “books”
5. The table contains the following 7 columns
   a. Id(Primary key), title, author, isbn, published_year, pages, availability
6. Insert 6 sample records into the books table.
7. Create an application user named “user” and set the password as “Library@123”.
8. Grant all privileges on the user.

3. Backend Project Setup

1. Create a project directory called “LibrarySystem” and navigate to it.
2. Initialize Node.js project. Open a new terminal and run the following command. `npm init -y`
3. Install dependencies

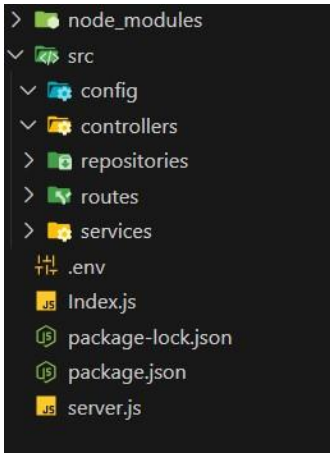
   ```
   npm install express mysql2 dotenv npm
   install --save-dev nodemon
   ```
4. Create project structure `mkdir src cd src mkdir config repositories services controllers routes cd ..`
5. Create the following files in the root directory,
   a. .env
   b. Server.js
   c. Index.js
6. Create .env file with the following content

```
PORT=5000
DB_HOST=localhost
DB_USER=user
DB_PASSWORD=Library@123
DB_NAME=library_db
DB_PORT=3306
```

7. Modify the “script” section package.json.

```
"scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
},
```

8. Final backend structure



4. Database connection

1. Create src/config/database.js file and create database configurations

```
const mysql = require('mysql2'); require('dotenv').config();

// Create connection pool const pool =
mysql.createPool({     host:
process.env.DB_HOST,     user:
process.env.DB_USER,     password:
process.env.DB_PASSWORD,     database:
process.env.DB_NAME,     port:
process.env.DB_PORT,
waitForConnections: true,
connectionLimit: 10,     queueLimit: 0
});

// Test connection
pool.getConnection((err, connection) => {
if (err) {
        console.error('X Database connection failed:', err.message);
process.exit(1);
    }
    console.log('✓ Database connected successfully');
connection.release();
});

// Export promise-based pool module.exports
= pool.promise();
```

5. Data access layer implementation

Data Access Layer (DAL) is a pattern that,

- Separates database operations from business logic
- Contains ONLY SQL queries
- Make code easier to maintain and test

1. Create src/repositories/bookRepository.js file

```
/**
* Book Data Access Layer
* Contains ONLY database queries - NO business logic
 */

const db = require('../config/database');

class BookRepository {

    //Get all books from database     async findAll() {
try {             const query = 'SELECT * FROM books ORDER BY id
DESC';             const [rows] = await db.query(query);
return rows;          } catch (error) {
            console.error('DAL Error - findAll:', error);
            throw new Error('Database error while fetching books');
        }
    }
}

// Export single instance
module.exports = new BookRepository();
```

6. Service layer implementation

Service Layer,

- Contains business logic and validation
- Calls DAL methods
- Sits between Controller and DAL

1. Create a file src/services/bookService.js

```
/**
* Book Service Layer
* Contains business logic and validation
 */
const bookRepository = require('../repositories/bookRepository');

class BookService {

    // Get all books     async getAllBooks() {
try {             const books = await
bookRepository.findAll();

return {
success: true,
data: books,
            };
        } catch (error) {
            console.error('Service Error - getAllBooks:', error);
throw error;
        }
    }
}

// Export single instance module.exports
= new BookService();
```

7. Controller layer implementation

Controller Layer,

- Handles HTTP requests and responses
- Calls service methods
- Returns JSON to client

1. Create src/controllers/bookController.js file

```
/**
* Book Controller
* Handles HTTP requests and responses
 */
```

```
const bookService = require('../services/bookService');

class   BookController   {
// GET /api/books
    async getAllBooks(req, res) {         try {
const result = await bookService.getAllBooks();
res.json(result);          } catch (error) {
            console.error('Controller Error:', error);
res.status(500).json({                 success: false,
message: 'Error fetching books',
error: error.message
            });
        }
    }
}

// Export single instance
module.exports = new BookController();
```

8. Routes and Server Setup

1. Create src/routes/books.js and create routes.

```
const express = require('express'); const router = express.Router();
const bookController = require('../controllers/bookController');

// CRUD routes
router.get('/', bookController.getAllBooks.bind(bookController));

module.exports = router;
```

2. Create main server file (server.js) in the root directory.

```
const express = require('express'); require('dotenv').config();

const app = express(); const PORT = process.env.PORT || 5000;
```

```
// Middleware
app.use(express.json()); app.use(express.urlencoded({
extended: true }));

// Request logging
app.use((req, res, next) => {
console.log(`${req.method} ${req.path}`);     next();
});

// Routes
app.use('/api/books', require('./src/routes/books.js'));
// Health check
app.get('/api/health', (req, res) => {
res.json({          status: 'OK',
message: 'Server is running',
timestamp: new Date().toISOString()
    });
});

// Root route
app.get('/', (req, res) => {     res.json({
message: 'Library API with Data Access Layer',
endpoints: {             books: '/api/books',
health: '/api/health'
        }
    });
});

// 404 handler app.use((req, res)
=> {     res.status(404).json({
success: false,         message:
'Route not found'
    });
});

// Start server
```

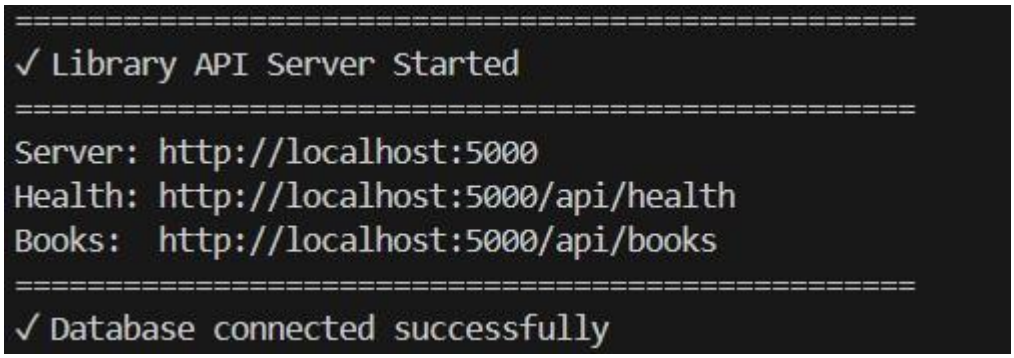
```
app.listen(PORT, () => {     console.log('='.repeat(50));
    console.log('✓       Library       API       Server       Started');
console.log('='.repeat(50));                          console.log(`Server:
http://localhost:${PORT}`);                            console.log(`Health:
http://localhost:${PORT}/api/health`);                  console.log(`Books:
http://localhost:${PORT}/api/books`);       console.log('='.repeat(50));
});
```

9. Testing with Postman

1. Start the server

   ```
   npm run dev
   ```

   You should see,

2. Open Postman
3. Create a new collection called “Library API”
4. Test endpoint
   GET - http://localhost:5000/api/books

10. Understanding the architecture

Client (Postman)
↓
Controller (bookController.js) - Handles HTTP
↓
Service (bookService.js) - Business Logic
↓
Repository (bookRepository.js) - Database Queries(DAL)
↓
Database (MySQL)